

Desarrollo de componentes de software

Proyecto: Sistema SVA (Prevención de Lavado de Dinero)

Conceptos Generales de Arquitectura Web

Para el desarrollo de aplicaciones modernas e institucionales, se utiliza una **Arquitectura Cliente-Servidor Desacoplada**. Esto significa que la interfaz de usuario (Frontend) y la lógica de negocio (Backend) viven en ecosistemas separados y se comunican exclusivamente mediante el protocolo HTTP(S) a través de una API.

1. Frontend

El Frontend es el entorno de ejecución que vive en el navegador del cliente.

- Es una capa "insegura" por definición, ya que el código (HTML, CSS, JS) corre en la máquina del usuario y puede ser manipulado.
- Su objetivo no es procesar reglas de negocio ni acceder a la base de datos, sino consumir servicios (APIs), pintar la interfaz y manejar el estado de la sesión localmente.

2. Backend

El Backend es el motor central. Se construye típicamente bajo el patrón arquitectónico **MVC (Modelo-Vista-Controlador)** o como una **API RESTful**.

- **Rutas (Router):** Definen los "endpoints" o URL expuestas (ej. GET /api/operaciones).
- **Controladores:** Son el "cerebro". Reciben la petición, extraen los parámetros, procesan la lógica algorítmica (ej. generar un archivo, validar un cálculo) y formatean la respuesta en JSON.
- **Modelos:** Se encargan exclusivamente de la comunicación con la Base de Datos. Ejecutan las queries SQL (ej. SELECT, INSERT) parametrizadas para evitar inyecciones.
- **Middlewares:** Interceptores de red que se ejecutan antes del controlador. Son cruciales para la seguridad, validando encabezados, tokens de autenticación o saneando el "body" de la petición.

3. Seguridad y Modelo Zero Trust

La regla de oro de la ingeniería web es: *El Backend jamás confía en el Frontend.*

- **JSON Web Tokens (JWT):** Un estándar para la transmisión segura de información. Al hacer login, el servidor emite un JWT firmado criptográficamente. El Frontend debe enviarlo en el *Header* (Authorization: Bearer <token>) en cada petición subsecuente.
- **Validación de Sesión:** Si alguien ataca la API directamente (ej. usando Postman) sin un token, el middleware rechaza la petición con un 401 Unauthorized. Si el token fue manipulado, retorna 403 Forbidden.
- **RBAC (Role-Based Access Control):** Autorización basada en roles. No basta con estar autenticado; el sistema debe verificar si el rol del usuario (embebido en el JWT) tiene

privilegios de lectura (GET) o de escritura (POST/PUT/DELETE) sobre un recurso específico.

Contribuciones Individuales al Sistema SVA

1. Refactorización de la Arquitectura de Base de Datos

- **Deconstrucción del Modelo Relacional:** Migré el sistema de un enfoque de reglas planas y globales a un modelo de **perfilamiento dinámico**. Diseñé la integridad referencial (Foreign Keys, ON DELETE CASCADE) para vincular tablas de clientes con reglas individuales, permitiendo una evaluación de riesgo algorítmica y asimétrica.
- **Corrección de Tipos de Datos para Almacenamiento:** Identifiqué y resolví errores de límite de caracteres (varchar(255)) al almacenar archivos estructurales. Modifiqué la tabla reportes_regulatorios utilizando tipos de dato TEXT para permitir el almacenamiento de la estructura completa de archivos XML sin saturar la memoria del servidor.

2. Desarrollo Backend (Node.js & Express)

- **Implementación del Patrón MVC:** Estructuré el backend separando estrictamente las responsabilidades en carpetas de /models, /controllers y /routes, facilitando la escalabilidad del proyecto.
- **Motor de Generación de XML (Reportes CNBV):** Programé desde cero el controlador reporteController.js. Este algoritmo:
 1. Recibe el cliente_id y alerta_id.
 2. Ejecuta un JOIN para extraer el perfil del cliente y RFC.
 3. Forja dinámicamente un archivo XML en formato *String* conforme a las regulaciones de la CNBV.
 4. Guarda el reporte estructural directamente en la base de datos PostgreSQL.
- **Cruces de Información Relacional:** Escribí las sentencias SQL en los modelos para construir vistas complejas, permitiendo unir operaciones, contratos y clientes mediante LEFT JOIN.

3. Desarrollo Frontend (Vanilla JavaScript)

- **Vistas Master-Detail y Deep Linking:** Implementé la lógica para que las pantallas respondieran dinámicamente a parámetros URL (?cliente_id=X), permitiendo flujos de trabajo donde un clic en una tabla de operaciones abre automáticamente modales con el contexto del cliente.
- **Manejo de Blobs para Descargas:** Desarrollé la funcionalidad en reportes.js para extraer el String del XML desde la base de datos, convertirlo en un objeto Blob tipo application/xml y forzar su descarga directa en el navegador del cliente sin necesidad de crear archivos físicos en el servidor.
- **Cliente API Centralizado:** Consolidé las peticiones fetch para incluir automáticamente el token de autorización desde el LocalStorage.

4. Implementación de Seguridad (Ciberseguridad en API)

- **Blindaje de Rutas con JWT:** Desarrollé e inyecté el middleware verificarToken en **todas** las rutas sensibles (/api/alertas, /api/operaciones, /api/reportes). Con esto, aseguré que la API sea impermeable a ataques de acceso directo (Broken Access Control) devolviendo códigos de error HTTP estrictos (401 y 403).
- **Control de Acceso Basado en Roles (RBAC):** Implementé un segundo middleware (verificarRolAdmin) para diferenciar entre perfiles "Admin" y "Visor". Los visores pueden consumir rutas GET, pero son bloqueados al intentar interactuar con mutaciones de estado (POST, PUT, DELETE).
- **Mitigación de Vulnerabilidades (OWASP):**
 - Prevención de **Inyección SQL** asegurando el uso exclusivo de consultas parametrizadas (pg).
 - Defensa contra **Verb Tampering** limitando explícitamente los verbos aceptados en el router de Express.
 - Validación de *Payloads* para evitar caídas del servidor ante cuerpos de petición nulos o malformados (Protección contra DoS a nivel de aplicación).

Todas estas evidencias se pueden encontrar en el branch "dev" en el repositorio del proyecto, donde se encuentra la versión estable más reciente.

<https://github.com/iRodama05/Finacial-Sentinel/tree/dev>